Регистрация на десктопе (Аэрофлот Бонус) в BPMN

Моделирование ПО · процесс регистрации в личном кабинете, десктопная версия

Этап 1. Кто участвует и границы процесса

Я описал, как человек регистрируется на сайте Аэрофлот Бонус с компьютера. В процессе четыре участника (в BPMN они называются «пулы»):

Участник	Что делает
Пользователь	Заполняет форму мышью и клавиатурой, вводит код из SMS.
Веб-система Аэрофлот	Сам сайт. Внутри три дорожки: **UI/UX** — то, что видит пользователь; **AuthService** — сервер, который делает проверки; **DataStore** — хранение данных.
SMS-шлюз	Внешняя система, которая присылает SMS с кодом.
База данных	Хранит пользователей, коды и согласия.

Границы процесса. Старт — пользователь открыл страницу регистрации. Конец — он успешно вошёл в личный кабинет или получил финальное сообщение об ошибке.

Глоссарий

Термин	Что это значит
Валидация на лету	Поле проверяется сразу, пока человек заполняет форму, а не после отправки.
Автофокус	Курсор сам становится в первое поле, кликать не нужно.
Tab-навигация	Переход между полями клавишей Tab.
Подсвечивание поля	Поле с ошибкой выделяется, например красной рамкой.
Блокировка кнопки отправки	Кнопка «Отправить» не нажимается, пока в форме есть ошибки.

Этап 2. Основная диаграмма

Читать диаграмму нужно слева направо: сверху — пользователь, ниже — сайт (три дорожки), потом SMS-шлюз и база данных. Файл для редактора BPMN: `registration_compact.bpmn`.

Что нарисовано

- **Задачи** (прямоугольники) — действия: открыть форму, автофокус, ввод данных, проверка при потере фокуса (on blur), ошибка под полем, блокировка кнопки, проверка email, создание кода, отправка SMS, сохранение в БД.
- **Шлюзы** (ромбы). С крестиком — выбор «да или нет»: «Поле валидно?», «Все поля валидны?», «Email уникален?», «Код верный?». С плюсом — две проверки одновременно (формат email и его уникальность).
- **Сообщения** (пунктир) — обмен между участниками, например сайт просит SMS-шлюз прислать код. Внутри сайта между дорожками обычные стрелки: так правильно по BPMN, пунктир бывает только между разными пулами.
- **Данные** (листочки): форма регистрации, объект пользователя, код подтверждения, документ согласия.
- **Граничные события** (кружки на краю задачи): таймаут кода и ошибка SMS-шлюза.

Пять сценариев

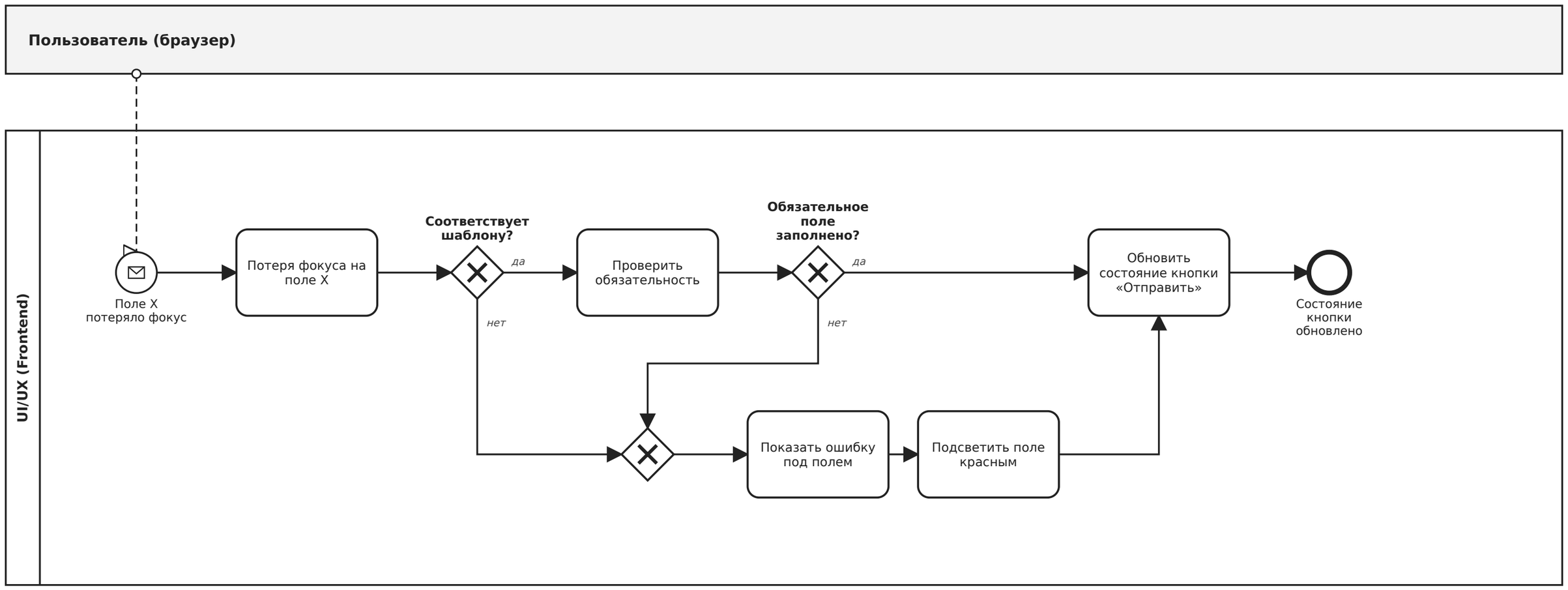
1. **Всё хорошо.** Поля верны, кнопка разблокирована, email свободен, приходит SMS, код введён верно, пользователь сохранён, вход в личный кабинет.
2. **Ошибка в поле** (формат, спецсимволы, слишком коротко). Под полем появляется ошибка, кнопка блокируется.
3. **Email уже есть** (ответ сервера 409). Под полем сообщение, что email занят.
4. **Неверный код.** Можно ввести ещё раз; когда попытки кончились — финальная ошибка.
5. **Код просрочен.** Срабатывает таймер, человек нажимает «Запросить новый код» и получает новое SMS.

Этап 3. Две небольшие диаграммы

Два места из основной схемы разобраны подробнее — проверка полей и согласия.

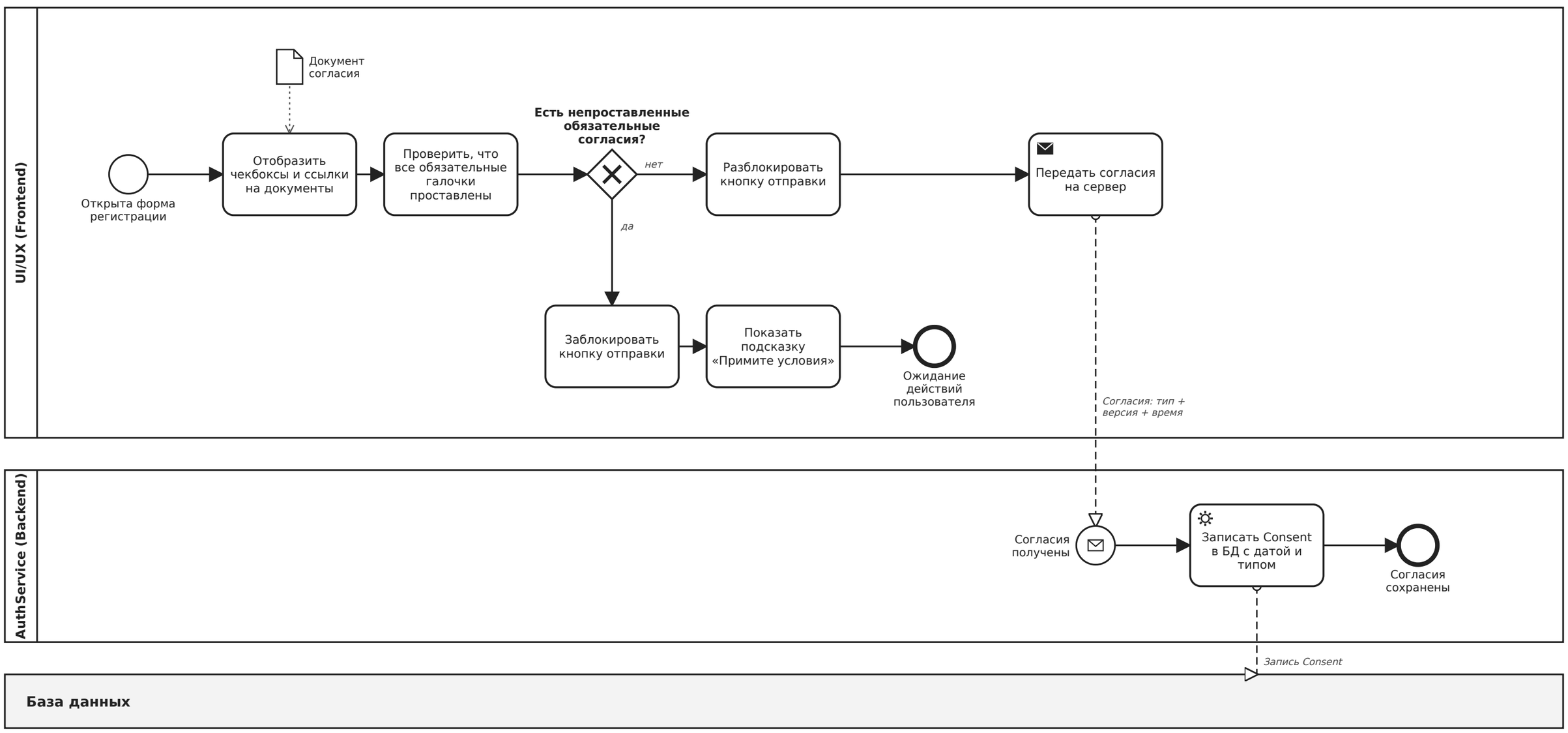
Проверка полей (on blur)

Когда человек уходит из поля, сайт проверяет, подходит ли значение по формату. Если нет — показывает ошибку под полем и подсвечивает поле красным. Если да — проверяет, что обязательное поле не пустое. В конце всегда обновляется состояние кнопки «Отправить». При нажатии «Отправить» проверка такая же, только для всех полей сразу. Файл: `validation_simple.bpmn`.



Согласия

Сайт показывает галочки и ссылки на документы и проверяет, все ли обязательные поставлены. Если нет — кнопка блокируется и появляется подсказка «Примите условия». Если всё отмечено — кнопка разблокируется, согласия уходят на сервер, и там их записывают в базу данных с датой и типом. Файл: `consents_simple.bpmn`.



Этап 4. Требования и вывод

Нефункциональные требования

Это требования не к тому, «что делает» сайт, а к тому, «как» он работает.

1. **Доступность.** У каждого поля есть подпись, текст ошибки хорошо виден, форму можно заполнить только с клавиатуры.
2. **Скорость.** Проверка поля работает сразу, без перезагрузки страницы.
3. **Безопасность.** Пароль хранят не как есть, а в виде хэша (password_hash).
4. **Юридическое.** Согласия сохраняют в базе с датой и типом.
5. **Понятные ошибки.** Если SMS не работает, пользователь видит понятное сообщение.

Связь требований с диаграммой

Требование	Где на диаграмме
1. Доступность	Задачи «Открыть форму регистрации» (подписи полей), «Показать сообщение об ошибке под полем», «Ввод данных (с Tab-навигацией)»
2. Скорость	Задача «Валидация поля при потере фокуса (on blur)» и шлюз «Поле валидно?»
3. Безопасность	Задача «Сохранить пользователя в БД» (пароль как password_hash)
4. Юридическое	Задача «Записать Consent в БД с датой и типом» (диаграмма согласий)
5. Понятные ошибки	Граничное событие «SMS-шлюз недоступен» → задача «Показать понятное сообщение, предложить повторить позже»

Граничные события

Граничное событие — это кружок на краю задачи. Он срабатывает, если что-то пошло не так, и процесс уходит по запасному пути.

Событие	Что происходит
1. Таймаут ожидания кода **часы на задаче «Ожидание ввода кода»**	Если человек не ввёл код вовремя, он нажимает «Запросить новый код», и сайт создаёт новый код и шлёт новое SMS.
2. Ошибка SMS-шлюза **молния на задаче «Отправить SMS через шлюз»**	Если шлюз недоступен, сайт показывает понятное сообщение «попробуйте позже», а процесс заканчивается финальной ошибкой. Пользователь в базе не создаётся.

Вывод

BPMN помогает просто и наглядно показать, кто и что делает при регистрации. Лучше всего на диаграмме видно проверку полей, блокировку кнопки и то, что происходит при ошибках. Фронтенд-разработчику это помогает понять, как должна вести себя форма. Бэкенд-разработчику — какие проверки и запросы нужны на сервере. UX-аналитику — увидеть, что каждое требование привязано к конкретной задаче и не забыто. Ещё один плюс: диаграмму можно открыть в редакторе BPMN и поправить.